



Name:	1. Bugtong, Arjay S. 2. Cagay-an, Juster Chiqui M.	Date:02/26/2025	Section:IDB1
Self-Guided Laboratory Activity: Building an Angular App with Routing and GitHub Collaboration		Activity 4b	Score:

This activity is designed to be self-paced and encourages students to learn through hands-on experience and collaboration.

Objective:

- Understand and implement different Angular v.19 routing methods.
- Collaborate to build a simple Angular app with multiple routes.
- Use GitHub for version control and collaboration.
- Submit the completed project to a GitHub repository with descriptive commit messages.

Materials Needed:

- Computers with Angular CLI installed
- Code editor (e.g., Visual Studio Code)
- Internet access for documentation and resources
- GitHub accounts for each student

Pre-Lab Preparation:

- Group by pairs.
- Ensure all students have Node.js, Angular CLI, and Git installed on their computers.
- One of the students in the pair will create a GitHub repository for the project and invite his/her partner to be a collaborator.
- Choose a routing method to create, merge your work, commit and push to GitHub.

Instructions:

1. Create a GitHub Repository and Add Collaborators:

- One student creates a new repository on GitHub:
 1. Go to GitHub and log in.
 2. Click on the "+" icon in the top right corner and select "New repository".
 3. Name the repository (lastname1-lastname2-angular-routing-lab), add a description, and click "Create repository".
- Add the other student as a collaborator:
 1. Go to the repository page.
 2. Click on "Settings" > "Collaborators".
 3. Add the GitHub username of the other student and click "Add collaborator".

2. Create a new Angular project.

```
ng new angular-routing-lab
cd angular-routing-lab
```

3. Initialize Git:

- In the project directory, initialize Git and connect to your GitHub repository:

```
git init
git remote add origin <repository-url>
```

4. Creating Components (10 minutes):

- Create three components: Home, About, and Contact. Use the following commands:

```
ng generate component home
ng generate component about
```

```
ng generate component contact
```

5. Setting Up Routes (15 minutes):

- Open the `app-routing.module.ts` file and define the routes for the components:

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { HomeComponent } from '../home/home.component';
import { AboutComponent } from '../about/about.component';
import { ContactComponent } from '../contact/contact.component';

const routes: Routes = [
  { path: '', redirectTo: '/home', pathMatch: 'full' },
  { path: 'home', component: HomeComponent },
  { path: 'about', component: AboutComponent },
  { path: 'contact', component: ContactComponent }];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule { }
```

6. Adding Navigation Links (10 minutes):

- In the `app.component.html` file, add navigation links to the components:

```
<nav>
  <a routerLink="/home">Home</a>
  <a routerLink="/about">About</a>
  <a routerLink="/contact">Contact</a>
</nav>
<router-outlet></router-outlet>
```

7. Implementing Child Routes (20 minutes):

- Create a new component for a child route, e.g., `Profile` under `About`:

```
ng generate component about/profile
```

- Update the `app-routing.module.ts` to include child routes:

```
const routes: Routes = [
  { path: '', redirectTo: '/home', pathMatch: 'full' },
  { path: 'home', component: HomeComponent },
  { path: 'about', component: AboutComponent, children: [
    { path: 'profile', component: ProfileComponent }
  ]},
  { path: 'contact', component: ContactComponent }
];
```

8. Lazy Loading Modules (30 minutes):

- Create a new module for lazy loading, e.g., `Admin`:

```
ng generate module admin --route admin --module app.module
```

- This command automatically sets up lazy loading for the `Admin` module.

9. Route Guards (30 minutes):

- Create a route guard to protect the `Admin` route:

```
ng generate guard admin/admin
```

- o Implement the guard logic in admin.guard.ts:

```
import { Injectable } from '@angular/core';
import { CanActivate, ActivatedRouteSnapshot, RouterStateSnapshot,
  UrlTree } from '@angular/router';
import { Observable } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class AdminGuard implements CanActivate {
  canActivate(
    next: ActivatedRouteSnapshot,
    tree: UrlTree | Promise<boolean | UrlTree> | boolean | UrlTree {
      // Add your authentication logic here
      return true; // Change this to actual authentication check
    }
  )
}

Update the app-routing.module.ts to use the guard:
const routes: Routes = [
  { path: '', redirectTo: '/home', pathMatch: 'full' },
  { path: 'home', component: HomeComponent },
  { path: 'about', component: AboutComponent, children: [
    { path: 'profile', component: ProfileComponent }
  ]},
  { path: 'contact', component: ContactComponent },
  { path: 'admin', loadChildren: () =>
import('./admin/admin.module').then(m => m.AdminModule), canActivate:
[AdminGuard] }
];
```

10. Testing the Application (20 minutes):

- o Run the application using the following command:

```
ng serve
```

- o Open a web browser and navigate to <http://localhost:4200>. Test the navigation links and routes to ensure everything works correctly.

11. Submitting to GitHub (15 minutes):

- o Initialize a Git repository in the project directory:

```
git init
git add .
git commit -m "Initial commit"
Push the project to a GitHub repository:
git remote add origin <your-github-repo-url>
git push -u origin master
```

Learning Outcomes:

- Students will understand and implement different Angular v.19 routing methods.
- Students will collaborate to build a functional Angular app with multiple routes.
- Students will submit their completed project to a GitHub repository.

Here are the expected outputs for each routing method in the activity:

1. Basic Routing

Expected Output:

APPDEV1

- The application should navigate between the Home, About, and Contact components using the defined routes.
- The navigation links in the app.component.html should work correctly, allowing users to switch between the different components.
- The URL should update accordingly when navigating to /home, /about, and /contact.

2. Child Routes

Expected Output:

- The About component should have a nested route for the Profile component.
- Navigating to /about/profile should display the Profile component within the About component.
- The URL should update to reflect the nested route structure.

3. Lazy Loading Modules

Expected Output:

- The Admin module should be lazy-loaded when navigating to the /admin route.
- The Admin component should be displayed when navigating to /admin.
- The application should only load the Admin module when the /admin route is accessed, improving the initial load time of the application.

4. Route Guards

Expected Output:

- The Admin route should be protected by the AdminGuard.
- The guard logic should determine whether the user can access the /admin route.
- If the guard condition is not met, the user should be redirected or prevented from accessing the Admin component.

Summary of Expected Outputs:

- 1. Basic Routing:**
 - Functional navigation between Home, About, and Contact components.
 - Correct URL updates for each route.
- 2. Child Routes:**
 - Nested routing within the About component.
 - Correct URL updates for nested routes.
- 3. Lazy Loading Modules:**
 - Lazy-loaded Admin module.
 - Improved initial load time by loading the Admin module only when needed.
- 4. Route Guards:**
 - Protected Admin route with guard logic.

Reflection Questions:

- 1. Technical Challenges:**
 - What specific technical challenges did you encounter while implementing the routing methods, and how did you resolve them?
 - First there is no module anything in our angular project so we had to search where to put the code. For example, app-routing.module.ts does not exist since apparently our project is a standalone therefore we put it in app.routes.ts and it did work
 - Were there any errors or bugs that were particularly difficult to debug? How did you approach solving them?

- It was hard debugging the directories since there are a lot of folders which contained the 'ts'. We literally had to look at it word for word to find the errors.
- How do child routes and lazy loading improve the structure and performance of an Angular application?
 - Child routes let you organize your app into smaller sections, like having subpages under a main page. Lazy loading makes the app faster by only loading the parts you need when you need them. Together, they keep your app neat and quick by loading less stuff upfront.
- What is the purpose of route guards, and how do they enhance the security of an application?
 - Route guards are like bouncers for your app, controlling who can access different pages. They check if a user is allowed to go to a route (like a profile page) before letting them in. This enhances security by blocking unauthorized access and making sure only the right people can reach sensitive areas or perform certain actions.
- How did collaborating with your peers help you understand Angular routing better?
 - Working with my peers really helped me get a better handle on Angular routing. We shared different approaches, tackled issues together, and I picked up some best practices along the way. Explaining things to them also helped me solidify what I understood.
- 2. **Learning Process:**
 - How did this hands-on activity help you understand Angular routing better compared to just reading or watching tutorials?
 - Barely helped us, it was easier to understand through tutorials.
 - What new concepts or techniques did you learn during this activity that you were not familiar with before?
 - I learned a lot about managing child routes, lazy loading, and how to set up route guards. Since our project was standalone, I also got to understand how to configure routes without the usual Angular module setup.
- 3. **Collaboration:**
 - How did working in a group influence your understanding of Angular routing? Did you find it helpful to discuss and solve problems together?
 - Working in a group really helped me get Angular routing. Talking through different approaches and solving problems together made things click faster. Explaining stuff to others also helped me understand it better. It felt less overwhelming and more collaborative.
 - What strategies did your group use to ensure effective collaboration and communication?
 - We made sure to communicate regularly through check-ins. We kept track of our progress on GitHub, which made it easy to share code and update each other. We also split tasks based on each person's strengths, so everyone could focus on what they did best.
- 4. **Application Design:**
 - How did you decide on the structure and design of your application? What factors influenced your design choices?
 - We kept the structure simple, focusing on what made the most sense for routing and modularity. We considered how users would navigate the app and organized the routes to make the flow intuitive.
 - How do you think the routing methods you implemented will impact the user experience of your application?
 - The routing methods will help make navigation smooth by keeping things organized and loading only the parts users need. This should make the app feel faster and more responsive.
- 5. **Best Practices:**
 - What best practices did you follow while setting up the routes and organizing your code?
 - We followed practices like keeping routes modular, using lazy loading where possible, and setting up guards for sensitive pages. We made sure the code was clean and easy to follow.
 - How did you ensure that your code is maintainable and scalable?
 - We kept the routing logic simple and organized, making it easy to add more routes as needed. Using lazy loading and guards helped keep the app scalable and efficient in the long run.
- 6. **Real-World Applications:**
 - Can you think of real-world applications or websites that use similar routing methods? How do they benefit from these methods?

- Many web apps like e-commerce sites or social media platforms use lazy loading and child routes. These methods help improve load times and organize content better, making the user experience smoother.
- o How would you apply what you learned in this activity to a real-world project or job?
 - In a real-world project, I'd use the same routing techniques to keep the app organized and efficient. I'd implement lazy loading for performance and route guards for security, ensuring a clean and secure user experience.

Grading Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs Improvement (1)
Basic Routing Implementation	All routes (Home, About, Contact) are correctly implemented and functional.	Most routes are correctly implemented and functional, with minor issues.	Some routes are correctly implemented, but there are noticeable issues.	Routes are incorrectly implemented or non-functional.
Child Routes Implementation	Child routes are correctly implemented and functional, with clear nested navigation.	Child routes are mostly correct, with minor issues in navigation.	Child routes are partially correct, but there are noticeable issues.	Child routes are incorrectly implemented or non-functional.
Lazy Loading Implementation	Lazy loading is correctly implemented for the Admin module, improving load time.	Lazy loading is mostly correct, with minor issues in implementation.	Lazy loading is partially correct, but there are noticeable issues.	Lazy loading is incorrectly implemented or non-functional.
Route Guards Implementation	Route guards are correctly implemented, effectively protecting routes.	Route guards are mostly correct, with minor issues in protection logic.	Route guards are partially correct, but there are noticeable issues.	Route guards are incorrectly implemented or non-functional.
Code Quality	Code is well-organized, clear, and follows best practices.	Code is mostly well-organized and clear, with minor issues.	Code is somewhat organized, but there are noticeable issues.	Code is poorly organized and unclear.
GitHub Collaboration	Regular commits with descriptive messages, active collaboration.	Regular commits with some descriptive messages, moderate collaboration.	Infrequent commits, minimal collaboration.	Few or no commits, lack of collaboration.
Project Functionality	All routes and components function as expected.	Most routes and components function as expected.	Some routes and components function as expected.	Many routes and components do not function as expected.
Reflection Responses	Reflection responses are insightful, well-written, and demonstrate a deep understanding.	Reflection responses are clear and demonstrate a good understanding.	Reflection responses are somewhat clear but lack depth.	Reflection responses are unclear or demonstrate a lack of understanding.
Collaboration and Participation	Actively participates in group discussions and contributes meaningfully to the project.	Participates in group discussions and contributes to the project.	Participates in group discussions but with limited contributions.	Rarely participates in group discussions and contributes minimally.

Additional Criteria:

1. **Error Handling and Debugging:**
 - o **Excellent (4):** Effectively handles errors and debugging, ensuring the application runs smoothly.
 - o **Good (3):** Handles most errors and debugging, with minor issues.
 - o **Satisfactory (2):** Handles some errors and debugging, but there are noticeable issues.
 - o **Needs Improvement (1):** Fails to handle errors and debugging, resulting in a non-functional application.
2. **Documentation and Comments:**
 - o **Excellent (4):** Code is well-documented with clear comments explaining the logic.
 - o **Good (3):** Code is mostly documented, with some comments explaining the logic.

APPDEV1

- **Satisfactory (2):** Code has limited documentation and comments.
- **Needs Improvement (1):** Code lacks documentation and comments.

Scoring:

- **Excellent:** 36–40 points
- **Good:** 28–35 points
- **Satisfactory:** 20–27 points
- **Needs Improvement:** 12–19 points

Answer Sheets:

